The Unattended Sheet

One flag turns a conversation into a command that runs without you.

START WITH SOMETHING THAT ONLY READS

Terminal

```
claude -p "Summarise what changed in this project in the
last week" --allowedTools "Read,Grep,Glob"
```

Then leave a trace

```
claude -p "Summarise what changed this week" \
  --allowedTools "Read,Grep,Glob" \
  > reports/weekly-$(date +%F).md
```

It reads standard input, so it fits what you already have

```
git diff main | claude -p "List anything in this diff
that looks risky. One line each. If nothing, say
nothing."
```

A SKELETON THAT FAILS SAFELY

scripts/weekly-summary.sh

```
#!/bin/bash
set -euo pipefail

cd "$(dirname "$0")/.."
mkdir -p reports
OUT="reports/weekly-$(date +%F).md"

claude -p "Summarise what changed in this project in the
last week. Group by area. Flag anything unfinished." \
  --allowedTools "Read,Grep,Glob" \
  > "$OUT"

echo "Wrote $OUT"
```

`set -euo pipefail` is the line that matters. Without it a step can fail in the middle and the script still exits looking successful.

THE FLAGS THAT MATTER

FLAG	WHAT IT ACTUALLY PERMITS
`-p "..."`	Run once and stop. No session, no waiting for you
`--allowedTools "Read,Edit"`	Exactly these, nothing else
`--permission-mode acceptEdits`	Write files without asking
`--output-format json`	Structured output a script can read
`--bare`	Skip your local setup, so it runs the same anywhere

THE RULE FOR UNATTENDED RUNS

Allow the narrowest set of tools that lets the job finish. Nobody is watching, so a run with everything allowed is the one that quietly does something you did not want at three in the morning.

FOUR WAYS THEY FAIL QUIETLY

The main step never happened

Something needed permission and nobody was there to give it, so it was simply skipped. The classic first unattended failure. Name the tools in advance.

No output file

You were not watching, so the file is all you get. A run that leaves no trace cannot be debugged, and you will need to debug it.

Too many tools allowed

Allowing everything so it does not get stuck is how something unwanted happens with nobody to stop it.

The script hid its own failure

Without `set -euo pipefail` a broken step in the middle still exits 0, so whatever calls it thinks all is well.